RAG Pipeline

Core Concepts & Interview Guide

English & Deutsch

Built with: C# · Ollama · Qdrant · BM25 · RRF · Cross-Encoder · Mistral

1. RAG — Retrieval Augmented Generation

What is RAG? / *Was ist RAG?*

[EN] RAG is a technique that combines a search system with a language model. Instead of relying purely on an LLM's training data (which can be outdated or wrong), RAG first retrieves relevant documents from a knowledge base, then feeds them to the LLM as context. The LLM generates an answer grounded in real, current documents.

[DE] *RAG ist eine Technik die ein Suchsystem mit einem Sprachmodell kombiniert. Anstatt sich auf die Trainingsdaten eines LLM zu verlassen (die veraltet oder falsch sein können), ruft RAG zuerst relevante Dokumente aus einer Wissensdatenbank ab und füttert sie dem LLM als Kontext. Das LLM generiert eine Antwort die auf echten, aktuellen Dokumenten basiert.*

Why does RAG exist? / *Warum gibt es RAG?*

[EN] LLMs hallucinate. They confidently produce wrong information. RAG solves this by grounding every answer in retrieved documents. It also solves the knowledge cutoff problem: the LLM doesn't need to know recent facts because RAG retrieves them at query time.

[DE] *LLMs halluzinieren, sie produzieren selbstbewusst falsche Informationen. RAG löst das indem jede Antwort in abgerufenen Dokumenten verankert wird. Es löst auch das Wissens-Cutoff-Problem: das LLM muss keine aktuellen Fakten kennen weil RAG sie zur Abfragezeit abruft.*

Interview Q: Explain RAG in simple terms.

Answer (EN): RAG stands for Retrieval Augmented Generation. When a user asks a question, the system first searches a document database to find the most relevant passages. Those passages are given to an LLM as context, and the LLM generates an answer based only on that context, not from memory. This prevents hallucination and keeps answers grounded in real documents.

Antwort (DE): *RAG steht für Retrieval Augmented Generation. Wenn ein Benutzer eine Frage stellt, durchsucht das System zuerst eine Dokumentendatenbank nach den relevantesten Abschnitten. Diese Abschnitte werden einem LLM als Kontext gegeben, und das LLM generiert eine Antwort basierend nur auf diesem Kontext, nicht aus dem Gedächtnis. Das verhindert Halluzination.*

2. Embeddings & Vector Search

What is an Embedding? / *Was ist ein Embedding?*

[EN] An embedding is a list of numbers (a vector) that represents the meaning of a piece of text. The model converts any text into a fixed-size array of floating point numbers. Texts with similar meaning produce similar numbers, this is the foundation of semantic search.

[DE] *Ein Embedding ist eine Liste von Zahlen (ein Vektor) die die Bedeutung eines Textstücks repräsentiert. Das Modell wandelt jeden Text in ein Array von Fließkommazahlen fester Größe um. Texte mit ähnlicher Bedeutung erzeugen ähnliche Zahlen, das ist die Grundlage der semantischen Suche.*

```
"I love dogs"          → [0.91, 0.23, 0.45, ...]   768 numbers
"Dogs are my favorite" → [0.89, 0.21, 0.47, ...]   ← similar!
"Stock market crashed"  → [0.12, 0.87, 0.03, ...]   ← different
```

Why 768 dimensions? / *Warum 768 Dimensionen?*

[EN] The nomic-embed-text model produces 768-dimensional vectors. Each dimension captures a different aspect of meaning; topic, tone, grammar, relationships between concepts. More dimensions = more nuance. 768 is the specific design choice of this model. OpenAI's ada-002 uses 1536.

[DE] *Das nomic-embed-text Modell erzeugt 768-dimensionale Vektoren. Jede Dimension erfasst einen anderen Bedeutungsaspekt wie Thema, Ton, Grammatik, Beziehungen zwischen Konzepten. Mehr Dimensionen bedeutet mehr Nuancen. 768 ist die spezifische Designentscheidung dieses Modells.*

What is Cosine Similarity? / *Was ist Kosinus-Ähnlichkeit?*

[EN] Cosine similarity measures the angle between two vectors. A small angle means similar direction = similar meaning (score near 1.0). A large angle means different direction = different meaning (score near 0.0). We use cosine instead of subtraction because it is robust to different text lengths.

[DE] *Kosinus-Ähnlichkeit misst den Winkel zwischen zwei Vektoren. Ein kleiner Winkel bedeutet ähnliche Richtung = ähnliche Bedeutung (Score nahe 1,0). Ein großer Winkel bedeutet verschiedene Richtung = verschiedene Bedeutung (Score nahe 0,0).*

Interview Q: What is the difference between keyword search and semantic search?

Answer (EN): Keyword search matches exact words. 'Fever' won't find 'elevated temperature'. Semantic search converts both query and documents into vectors and compares their meaning using cosine similarity. So 'fever' does find 'elevated temperature' because they're semantically close. We use both in our hybrid search system.

Antwort (DE): *Keyword-Suche findet exakte Wörter. 'Fieber' findet nicht 'erhöhte Temperatur'. Semantische Suche wandelt Query und Dokumente in Vektoren um und vergleicht deren Bedeutung,'Fieber' findet 'erhöhte Temperatur' weil sie semantisch nah sind. Wir verwenden beide in unserem hybriden Suchsystem.*

3. Chunking — Splitting Documents

What is Chunking? / *Was ist Chunking?*

[EN] Chunking splits a large document into smaller pieces before embedding. Embedding models have a token limit, you cannot feed an entire PDF at once. Each chunk becomes independently searchable. The art is in choosing chunk size and overlap correctly.

[DE] *Chunking teilt ein großes Dokument in kleinere Stücke auf bevor es eingebettet wird. Embedding-Modelle haben ein Token-Limit, man kann keine gesamte PDF auf einmal einspeisen. Jeder Chunk wird unabhängig durchsuchbar. Die Kunst liegt darin Chunk-Größe und Überlappung richtig zu wählen.*

Why overlap? / *Warum Überlappung?*

[EN] If you split text without overlap, an important sentence that spans a chunk boundary gets cut in half, losing its meaning. With overlap (e.g. 5 words), chunk N ends with the same 5 words that chunk N+1 starts with. No sentence gets lost at the edges.

[DE] *Wenn Text ohne Überlappung aufgeteilt wird, wird ein wichtiger Satz der eine Chunk-Grenze überspannt in der Mitte abgeschnitten. Mit Überlappung (z.B. 5 Wörter) endet Chunk N mit denselben 5 Wörtern mit denen Chunk N+1 beginnt.*

```
Text:    [word1 ... word40 | word41 ... word80]
Chunk 1: [word1 ......... word40]
Chunk 2:             [word36 ......... word75]  ← 5-word overlap
```

Interview Q: How did you decide on chunk size?

Answer (EN): We use 40-word chunks with 5-word overlap. The chunk size must fit within the embedding model's context limit. Smaller chunks give more precise retrieval but less context per result. Larger chunks give more context but noisier embeddings. 40 words is a practical balance for the nomic-embed-text model. Overlap ensures no sentence is cut at a chunk boundary.

Antwort (DE): *Wir verwenden 40-Wort-Chunks mit 5-Wort-Überlappung. Die Chunk-Größe muss innerhalb des Kontextlimits des Embedding-Modells liegen. Kleinere Chunks geben präziseres Retrieval aber weniger Kontext pro Ergebnis. 40 Wörter ist eine praktische Balance. Überlappung stellt sicher dass kein Satz an einer Chunk-Grenze abgeschnitten wird.*

4. BM25 — Keyword Search Algorithm

What is BM25? / *Was ist BM25?*

[EN] BM25 (Best Match 25) is a keyword search algorithm from the 1990s still used by Elasticsearch and Google today. It scores documents based on exact word matches, with two important adjustments: rare words score higher than common words (IDF), and the score is normalized for document length.

[DE] *BM25 (Best Match 25) ist ein Keyword-Suchalgorithmus aus den 1990ern der heute noch von Elasticsearch und Google verwendet wird. Er bewertet Dokumente anhand exakter Wortübereinstimmungen, mit zwei wichtigen Anpassungen: seltene Wörter erzielen höhere Scores als häufige (IDF), und der Score wird für die Dokumentlänge normalisiert.*

IDF — Inverse Document Frequency / *IDF — Inverse Dokumenthäufigkeit*

[EN] Rare words that appear in few documents score higher than common words that appear everywhere. If 'retrieval' appears in only 3 of 86 chunks, it's a strong signal. If 'the' appears in all 86, it tells us nothing.

[DE] *Seltene Wörter die in wenigen Dokumenten vorkommen erzielen höhere Scores als häufige Wörter die überall vorkommen. Wenn 'retrieval' nur in 3 von 86 Chunks vorkommt, ist das ein starkes Signal.*

```
IDF = log((N - df + 0.5) / (df + 0.5) + 1)
N = total chunks,  df = chunks containing this term
```

Length Normalization / *Längennormalisierung*

[EN] A short chunk where 'RAG' is the main topic scores higher than a long chunk that merely mentions 'RAG' once. BM25 uses k1=1.5 and b=0.75 as standard constants to balance term frequency against document length.

[DE] *Ein kurzer Chunk wo 'RAG' das Hauptthema ist, erzielt einen höheren Score als ein langer Chunk der 'RAG' nur einmal erwähnt. BM25 verwendet k1=1,5 und b=0,75 als Standardkonstanten.*

Interview Q: Why use BM25 when you already have vector search?

Answer (EN): Vector search is great for semantic similarity but can miss exact rare terms. Acronyms like 'SP-API' or specific model names. BM25 finds exact keyword matches that vector search might miss. They complement each other: vector search understands meaning, BM25 finds exact terms. Together in a hybrid system, they cover each other's weaknesses.

Antwort (DE): *Vektorsuche ist gut für semantische Ähnlichkeit, kann aber exakte seltene Begriffe verpassen. Akronyme wie 'SP-API' oder spezifische Modellnamen. BM25 findet exakte Keyword-Übereinstimmungen die Vektorsuche verpassen könnte. Zusammen in einem hybriden System decken sie gegenseitig ihre Schwächen ab.*

5. Hybrid Search & Reciprocal Rank Fusion

What is Hybrid Search? / *Was ist hybride Suche?*

[EN] Hybrid search runs both vector search and BM25 simultaneously, then merges the results. Neither system alone is perfect: vector search understands meaning but misses exact terms; BM25 finds exact terms but misses synonyms. Combined, they give much better results.

[DE] *Hybride Suche führt sowohl Vektorsuche als auch BM25 gleichzeitig aus und fügt dann die Ergebnisse zusammen. Kein System allein ist perfekt: Vektorsuche versteht Bedeutung aber verfehlt exakte Begriffe; BM25 findet exakte Begriffe aber verfehlt Synonyme.*

What is RRF? / *Was ist RRF?*

[EN] Reciprocal Rank Fusion is the formula that merges two ranked result lists fairly. It ignores actual scores (which are incomparable between systems) and only uses rank positions. Each result gets a score of 1/(60+rank) from each system. Results appearing in both lists receive double credit and float to the top.

[DE] *Reciprocal Rank Fusion ist die Formel die zwei Ranglisten fair zusammenführt. Sie ignoriert tatsächliche Scores (die zwischen Systemen nicht vergleichbar sind) und verwendet nur Rangpositionen. Jedes Ergebnis erhält einen Score von 1/(60+Rang) von jedem System.*

```
score = 1 / (60 + rank)
Rank 1 → 1/61 = 0.01639
Rank 5 → 1/65 = 0.01538   ← small gap, neither system dominates
```

Why 60 in the formula? / *Warum 60 in der Formel?*

[EN] The constant 60 prevents rank 1 from dominating too strongly over lower ranks. It was found empirically in the original RRF paper (Cormack et al. 2009) by testing across dozens of benchmarks. It has become the standard default used by Elasticsearch, Weaviate, and most RAG frameworks.

[DE] *Die Konstante 60 verhindert dass Rang 1 zu stark über niedrigere Ränge dominiert. Sie wurde empirisch im ursprünglichen RRF-Paper (Cormack et al. 2009) gefunden. Sie ist zum Standard-Default geworden der von Elasticsearch, Weaviate und den meisten RAG-Frameworks verwendet wird.*

Interview Q: How do you combine results from two different search systems?

Answer (EN): We use Reciprocal Rank Fusion. Both vector search and BM25 return ranked lists. Instead of comparing their raw scores which use different scales, RRF converts each rank to 1/(60+rank) and sums the scores. Results found by both systems receive contributions from both and naturally rise to the top. A result marked 'Vector + BM25' has the highest confidence.

Antwort (DE): *Wir verwenden Reciprocal Rank Fusion. Beide Systeme geben Ranglisten zurück. Anstatt rohe Scores zu vergleichen, die unterschiedliche Skalen verwenden, konvertiert RRF jeden Rang zu 1/(60+Rang) und summiert die Scores. Ergebnisse die von beiden Systemen gefunden werden steigen natürlich nach oben.*

6. Reranking — Cross-Encoder

What is a Reranker? / *Was ist ein Reranker?*

[EN] A reranker takes the top N candidates from retrieval and re-scores them with a more powerful but slower model. The retrieval step is fast but approximate. The reranker is slow but precise. We only rerank the top 5 results, not all 86 chunks to keep latency acceptable.

[DE] *Ein Reranker nimmt die Top-N-Kandidaten aus dem Retrieval und bewertet sie mit einem leistungsfähigeren aber langsameren Modell neu. Der Retrieval-Schritt ist schnell aber approximativ. Der Reranker ist langsam aber präzise. Wir reranken nur die Top-5-Ergebnisse, nicht alle 86 Chunks.*

Bi-Encoder vs Cross-Encoder / *Bi-Encoder vs Cross-Encoder*

[EN] A bi-encoder (used for vector search) embeds query and document separately then compares. Fast but loses nuance. A cross-encoder reads a query AND document together as a pair, 'does this specific chunk actually answer this specific question?' Much more accurate but 100x slower. That is why we use bi-encoder for retrieval and cross-encoder only for reranking.

[DE] *Ein Bi-Encoder (für Vektorsuche) bettet Query und Dokument separat ein und vergleicht dann. Schnell aber verliert Nuancen. Ein Cross-Encoder liest Query UND Dokument zusammen als Paar. Viel genauer aber 100x langsamer. Deshalb verwenden wir Bi-Encoder für Retrieval und Cross-Encoder nur für Reranking.*

```
Bi-Encoder:     Query → [embed] → A     Document → [embed] → B     score = cosine(A,B)
Cross-Encoder:  [CLS] Query [SEP] Document [SEP] → score
```

What is a Tokenizer? / *Was ist ein Tokenizer?*

[EN] A tokenizer converts text into numbers (token IDs) that a model can process. It splits text into subwords from a vocabulary of ~30,000 tokens. Special tokens [CLS], [SEP], [UNK] give the model structural signals. [CLS] marks the start; the model's output at [CLS] position = the relevance score.

[DE] *Ein Tokenizer wandelt Text in Zahlen (Token-IDs), um die ein Modell verarbeiten kann. Er teilt Text in Teilwörter aus einem Vokabular von ~30.000 Tokens auf. Spezielle Tokens [CLS], [SEP], [UNK] geben dem Modell Struktursignale.*

Interview Q: Why do you need a reranker if you already have hybrid search?

Answer (EN): Hybrid search is fast but approximate, it compares vectors and keywords independently. A cross-encoder reranker reads the query and each chunk together as a pair and scores how well the chunk actually answers the question. It catches cases where a chunk is semantically similar to the query but doesn't actually answer it. The reranker adds a precise final filter on top of the fast approximate retrieval.

Antwort (DE): *Hybride Suche ist schnell aber approximativ, sie vergleicht Vektoren und Keywords unabhängig. Ein Cross-Encoder Reranker liest Query und Chunk zusammen als Paar und bewertet wie gut der Chunk die Frage tatsächlich beantwortet. Er erkennt Fälle wo ein Chunk semantisch ähnlich ist aber die Frage nicht wirklich beantwortet.*

7. Grounded Generation — LLM Answer Generation

What is Grounded Generation? / *Was ist fundierte Generierung?*

[EN] Grounded generation means the LLM is instructed to answer ONLY from the provided context, not from its own training memory. This prevents hallucination. If the context doesn't contain the answer, the model says 'I cannot find this in the document' rather than making something up.

[DE] *Fundierte Generierung bedeutet dass das LLM angewiesen wird NUR aus dem bereitgestellten Kontext zu antworten, nicht aus seinem eigenen Trainingsgedächtnis. Das verhindert Halluzination. Wenn der Kontext die Antwort nicht enthält, sagt das Modell 'Ich kann das im Dokument nicht finden'.*

What is token streaming? / *Was ist Token-Streaming?*

[EN] Instead of waiting for the entire answer to generate (which can take minutes), streaming sends each token to the client as it is generated, exactly like ChatGPT typing in real time. We use Stream=true with await foreach in C# to process tokens as they arrive.

[DE] *Anstatt auf die gesamte Antwort zu warten (was Minuten dauern kann), sendet Streaming jeden Token an den Client sobald er generiert wird, genau wie ChatGPT das in Echtzeit tippt. Wir verwenden Stream=true mit await foreach in C#.*

What is a Prompt? / *Was ist ein Prompt?*

[EN] A prompt is the structured text input given to an LLM. Our RAG prompt has three parts: (1) Instruction: tells the model its role and constraints; (2) Context: the 3 reranked chunks; (3) Question: the user's query followed by 'Answer:' which the model continues from.

[DE] *Ein Prompt ist die strukturierte Texteingabe die einem LLM gegeben wird. Unser RAG-Prompt hat drei Teile: (1) Anweisung: sagt dem Modell seine Rolle; (2) Kontext: die 3 Reranked-Chunks; (3) Frage: die Benutzeranfrage gefolgt von 'Answer:'.*

Interview Q: How do you prevent the LLM from hallucinating?

Answer (EN): We use grounded generation, the prompt explicitly instructs the model to answer only from the provided context chunks, and to say 'I cannot find this in the document' if the context is insufficient. The model has no permission to use its own training knowledge. Combined with reranking to ensure high-quality context, this keeps answers factually grounded.

Antwort (DE): *Wir verwenden fundierte Generierung, der Prompt weist das Modell explizit an nur aus den bereitgestellten Kontext-Chunks zu antworten. Das Modell hat keine Erlaubnis sein eigenes Trainingswissen zu verwenden. Kombiniert mit Reranking für hochwertige Kontexte hält das Antworten faktisch fundiert.*

8. Full Pipeline — End to End

Complete flow when a user asks a question:

1. PDF loaded → text extracted page by page (PdfPig)
2. Text cleaned → ligatures fixed, URLs removed, spaces repaired (Regex)
3. Text chunked → 40-word overlapping pieces (Recursive Word Chunking)
4. Each chunk embedded → 768-dimensional vector (nomic-embed-text via Ollama)
5. Vectors stored in Qdrant Cloud with original text as payload

—— QUERY TIME ——

6. User query embedded → same 768D space
7. Vector search → top 5 by cosine similarity (Qdrant)
8. BM25 search → top 5 by keyword relevance (in-memory)
9. RRF merge → top 5 combined candidates (score = 1/(60+rank))
10. Cross-encoder reranking → top 3 truly relevant chunks
11. Mistral reads top 3 chunks → generates grounded answer (streaming)

9. Technology Stack

Technology	Purpose	Why chosen
C# / .NET 8	Main language	Enterprise-grade, strongly typed, great for AI tooling
Ollama	Local LLM runner	Free, no API key, runs nomic-embed-text + Mistral locally
nomic-embed-text	Embedding model	768D, open source, excellent quality for retrieval tasks
Qdrant Cloud	Vector database	Free tier, gRPC API, HNSW index, fast cosine search
PdfPig	PDF parsing	Pure C# PDF text extraction, no native dependencies
BM25 (custom)	Keyword search	Built from scratch, no library needed
RRF	Result merging	Simple, parameter-free, proven in research (2009)
ms-marco-MiniLM	Cross-encoder reranker	90MB, runs locally via ONNX, trained for passage ranking
Mistral 7B	Answer generation	Best small open-source chat model, runs locally
GitHub Actions	CI/CD	Auto-builds on every push